

GOFORGE

Technical Specification — Internal Architecture

Version 1.0 · April 2026 · Confidential

1. Overview

This document describes the internal architecture of the GoForge CLI tool. It is intended for engineers building or contributing to GoForge itself — not for users of the tool. For the DSL specification and user-facing features, see the GoForge Product Specification.

GoForge is written in Go. It is a single static binary with no runtime dependencies. It reads a `goforge.yaml` spec file and produces source code files on disk. The process is deterministic — identical input always produces identical output. No network calls, no AI, no side effects beyond writing files.

Repository structure

```
goforge/
  cmd/goforge/      ← CLI entrypoint (cobra commands)
  internal/
    spec/           ← SpecAST types, parser, validator, resolver
    plan/           ← Diff planner, GenerationPlan, Task types
    generator/       ← Generator interface + registry + orchestrator
    generators/      ← Individual generators (model, service, dto, ...)
      go/           ← Go-specific generators
      java/         ← Java-specific generators
      shared/       ← Language-agnostic generators (migration, types)
    template/       ← Template engine, FuncMap, ImportCollector
    emitter/        ← File writer, chmod management, lock file I/O
    imports/        ← ImportSet, ImportCollector, hierarchy validator
    diff/           ← Lock file schema, spec comparison, change detection
  templates/
    go/             ← *.go.tmpl files
    java/           ← *.java.tmpl files
    sql/            ← *.sql.tmpl files
    shared/         ← partials used across languages
  testdata/        ← golden file test fixtures
  Makefile
  go.mod
```

2. Core Data Structures

Understanding the data structures that flow through the pipeline is essential before reading any individual component. Each stage transforms one struct into another — nothing is shared via global state.

2.1 SpecAST

The raw output of the parser. Directly mirrors the YAML structure. All fields are exactly what the user wrote — no defaults filled in, no inference applied. String enums are still strings, relations are still unresolved names.

```
// internal/spec/ast.go

type SpecAST struct {
    Version      int
    Project      string
    Lang         string          // "go" | "java" — validated later
    Framework    string
    DB           string
    Config       []ConfigVarAST
    Resources    []ResourceAST
    Types        []CustomTypeAST
    Transactions []TransactionAST
    Auth         *AuthAST
    Messaging    *MessagingAST
    Externals    []ExternalAST
    Jobs         []JobAST
    Cache        *CacheAST
    Storage      *StorageAST
    Observability *ObservabilityAST
    Middleware   []string
    Extensions   []ExtensionAST
    Overrides    *OverridesAST
}

type ResourceAST struct {
    Name      string
    Table     string          // empty if not declared — Resolver fills it
    Fields    []FieldAST
    Relations []RelationAST
    Endpoints []EndpointAST
    Queries   []QueryAST
    Rules     *RulesAST
    States    *StateMachineAST
    Hooks     *HookDeclAST
    DTOs      []DTOAST
    Errors    []string
}

type FieldAST struct {
    Name      string
    Type      string          // "str" | "int" | "Money" | etc.
    Required  bool
}
```

```

    Unique      bool
    Nullable    bool
    Private     bool
    Default     interface{}
    Values      []string    // enum values
    Rules       []RuleAST
}

```

2.2 ResolvedSpec

The output of the Resolver. Every implicit value is now explicit. Types are resolved to their language-specific forms. Relations are wired to their target resources. Indexes are derived. This is the struct all generators receive — they do zero inference.

```

// internal/spec/resolved.go

type ResolvedSpec struct {
    // identity
    Project      string
    Module       string    // Go module path e.g.
    "github.com/user/orders-service"
    Lang         Lang      // typed enum, not string
    Framework    Framework
    DB           DBDriver

    // resolved content
    Config        []ResolvedConfigVar
    Resources     []ResolvedResource
    Types         TypeRegistry    // map[string]ResolvedCustomType
    Transactions  []ResolvedTransaction
    Auth          *ResolvedAuth
    Messaging     *ResolvedMessaging
    Externals     []ResolvedExternal
    Jobs          []ResolvedJob
    Cache         *ResolvedCache
    Overrides     ResolvedOverrides
}

type ResolvedResource struct {
    Name          string
    Table         string    // always set - never empty
    PackageName   string    // snake_case of Name
    Fields        []ResolvedField
    Relations     []ResolvedRelation
    Endpoints     []ResolvedEndpoint
    Queries       []ResolvedQuery
    Indexes       []ResolvedIndex    // derived by Resolver from queries
    Rules         ResolvedRules
    States        *ResolvedStateMachine
    Hooks         ResolvedHooks    // all hook points, including derived ones
    DTOs          []ResolvedDTO
    Mappers       []ResolvedMapper    // derived by Resolver from DTOs
    Errors        []ResolvedError
    HasSoftDelete bool
    HasTimestamps bool
}

```

```

type ResolvedField struct {
    Name          string
    ColumnName    string          // snake_case of Name
    Type          ResolvedType
    Required      bool
    Unique        bool
    Nullable      bool
    Private       bool
    Default       interface{}
    Rules         []ResolvedRule
    DBType        string          // "VARCHAR(255)", "BIGINT", "JSONB" etc.
    GoType        string          // "string", "int64", "types.Money" etc.
    JavaType      string          // "String", "Long", "Money" etc.
}

```

2.3 GenerationPlan

The output of the Diff Planner. A sorted list of Tasks. Each Task targets one generator with a specific context. The order encodes the dependency graph — tasks that depend on others always appear later.

```

// internal/plan/plan.go

type GenerationPlan struct {
    Tasks []Task
    Reason PlanReason // FirstRun | Update | AddResource
}

type Task struct {
    GeneratorID string          // e.g. "go.model", "go.service", "sql.migration"
    Context     GeneratorContext
    DependsOn   []string          // generator IDs that must complete first
    Priority     int              // lower = runs earlier within a tier
}

type GeneratorContext struct {
    Spec          *ResolvedSpec
    Resource       *ResolvedResource // nil for cross-cutting generators
    Changes       []SpecChange      // what changed since last lock (Update only)
    OutputDir     string
}

type SpecChange struct {
    Kind      ChangeKind // AddField | RemoveField | AddResource | AddEndpoint | ...
    Target    string      // resource name, field name, etc.
    Before    interface{} // previous value (nil on Add)
    After     interface{} // new value (nil on Remove)
}

```

2.4 File

The output of every generator. A simple path + content pair. The emitter writes these to disk. Generators never write to disk directly — they only return File values.

```
// internal/generator/file.go

type File struct {
    Path      string    // relative to project root, e.g. "generated/user/model.go"
    Content   []byte    // fully rendered, formatted source content
    ReadOnly  bool      // if true, emitter sets chmod 444 after writing
    Mode      fs.FileMode
}
```

3. Pipeline Stages

The pipeline is a linear sequence of transformations. Each stage has a single responsibility and a typed interface. No stage has side effects except the final emitter which writes files.

3.1 Parser

Reads goforge.yaml bytes and produces a SpecAST. Two sub-steps: the standard yaml.v3 library converts bytes to a generic map, then the GoForge parser maps that to typed structs using reflection and explicit field mapping.

```
// internal/spec/parser.go

type Parser struct{}

func (p *Parser) Parse(data []byte) (*SpecAST, error) {
    // Step 1: standard YAML parse → generic map
    var raw map[string]interface{}
    if err := yaml.Unmarshal(data, &raw); err != nil {
        return nil, &ParseError{Line: extractLine(err), Msg: err.Error()}
    }

    // Step 2: map generic map → typed SpecAST
    ast := &SpecAST{}
    if err := mapToAST(raw, ast); err != nil {
        return nil, err
    }
    return ast, nil
}

// mapToAST uses explicit field mapping — no reflection magic.
// Every field is mapped by name. Unknown fields produce warnings.
// Missing required fields are caught by the Validator, not the Parser.
```

Parser scope

The Parser does NOT validate. It does NOT fill defaults. It does NOT resolve types. Its only job is: bytes → SpecAST. Anything that requires knowledge of the spec schema goes in the Validator. Anything that requires cross-field inference goes in the Resolver.

3.2 Validator

Takes the SpecAST and checks all semantic constraints. Collects ALL errors in a single pass — never stops at the first error. Returns a []ValidationError with file line numbers.

What the validator checks

Category	Checks performed
Required fields	version, project, lang, framework, db, config, resources all present
Enum values	lang in [go,java], framework in [gin,echo,fiber,spring], db in [postgres,mysql,mongo]
Type references	Every field type is a known primitive or declared in types: block
Relation targets	belongs_to / has_many targets exist as resource names
FK field existence	fk: field name exists on the correct resource
DTO field refs	Every field name in a DTO exists on the resource
Private in DTO	private: true fields cannot appear in any DTO — hard error
Enum values list	type: enum always has non-empty values: list
State machine	states.field exists and is type: enum. All from/to values in enum values list
Auth references	roles on endpoints exist in auth.roles list
Duplicate names	No two resources, types, or externals share a name
Compute fields	compute: true only on response DTOs, never on request or event DTOs
Immutable fields	immutable: true only on update request DTOs

```
// internal/spec/validator.go

type Validator struct{}

func (v *Validator) Validate(ast *SpecAST) []ValidationError {
    var errs []ValidationError

    // collect - never return early on first error
    errs = append(errs, v.validateTopLevel(ast)...)
    errs = append(errs, v.validateResources(ast)...)
    errs = append(errs, v.validateTypes(ast)...)
    errs = append(errs, v.validateRelations(ast)...)
    errs = append(errs, v.validateDTOs(ast)...)
    errs = append(errs, v.validateAuth(ast)...)
    errs = append(errs, v.validateExternals(ast)...)

    return errs
}

type ValidationError struct {
    Line      int
    Field     string // e.g. "resources[0].fields[2].type"
    Message   string // human-readable description
    Code      string // machine-readable e.g. "PRIVATE_FIELD_IN_DTO"
}
```

3.3 Resolver

Takes the validated SpecAST and produces a ResolvedSpec. This is the richest and most complex stage. It fills in all defaults, derives all implicit values, builds the TypeRegistry, wires relations, and derives indexes from query declarations.

Resolver responsibilities in order

Step	What the Resolver does
1. Build TypeRegistry	Index all declared custom types by name for O(1) lookup during field resolution
2. Resolve lang/db	Convert string enums to typed Go constants (LangGo, DBPostgres etc.)
3. Resolve fields	For each field: fill table column name, fill DBType/GoType/JavaType from type map, resolve custom type references via TypeRegistry
4. Derive table names	If table: absent, set to snake_case plural of resource Name
5. Wire relations	Resolve belongs_to/has_many target names to ResolvedResource pointers. Add FK column to the correct resource.
6. Derive indexes	Walk all find_by, exists, count, sum, order_by declarations. Build minimum covering index set.
7. Expand soft delete	If soft_delete: true, add deleted_at field, modify all list/find queries to append AND deleted_at IS NULL
8. Expand timestamps	If overrides.timestamps: true (default), add created_at and updated_at to every resource
9. Resolve DTOs	Map DTO field references to ResolvedField. Derive mapper functions needed. Build hook stubs for compute fields.
10. Resolve hooks	Collect all hook points: lifecycle hooks + compute hooks + custom hooks + state change hooks
11. Derive DI graph	Determine constructor signature for each service based on its dependencies (repo, hooks, externals, messaging)
12. Validate hierarchy	Check import DAG — no circular dependencies. Error before any generation.

```
// internal/spec/resolver.go

type Resolver struct {
    typeMap map[string]map[string]string // lang → dslType → langType
    dbMap    map[string]string                    // dslType → dbType
}
```

```

func NewResolver() *Resolver {
    return &Resolver{
        typeMap: map[string]map[string]string{
            "go": {
                "str":      "string",
                "int":      "int64",
                "decimal":   "float64",
                "bool":      "bool",
                "timestamp": "time.Time",
                "uuid":      "uuid.UUID",
                "json":      "json.RawMessage",
            },
            "java": {
                "str":      "String",
                "int":      "Long",
                "decimal":   "BigDecimal",
                "bool":      "Boolean",
                "timestamp": "Instant",
                "uuid":      "UUID",
                "json":      "JsonNode",
            },
        },
        dbMap: map[string]string{
            "str":      "VARCHAR(255)",
            "int":      "BIGINT",
            "decimal":   "NUMERIC(10,2)",
            "bool":      "BOOLEAN",
            "timestamp": "TIMESTAMP TZ",
            "uuid":      "UUID",
            "json":      "JSONB",
        },
    }
}

func (r *Resolver) Resolve(ast *SpecAST) (*ResolvedSpec, error) {
    typeRegistry := r.buildTypeRegistry(ast)
    spec := &ResolvedSpec{}
    // ... resolve each section in order
    return spec, nil
}

```

3.4 Diff Planner

Compares the ResolvedSpec against the lock file to produce a GenerationPlan. On first run (no lock file) the plan contains all tasks. On update the plan contains only tasks affected by what changed.

```

// internal/diff/planner.go

type Planner struct {
    registry *generator.Registry
}

func (p *Planner) Plan(spec *ResolvedSpec, lock *LockFile) (*GenerationPlan, error) {
    {

```

```

    if lock == nil {
        return p.fullPlan(spec), nil    // first run — generate everything
    }

    changes := p.diffSpecVsLock(spec, lock)
    if len(changes) == 0 {
        return &GenerationPlan{}, nil    // nothing changed — nothing to do
    }
    return p.partialPlan(spec, changes), nil
}

// partialPlan: for each change, find all generators affected.
// A field addition affects: migration, model, dto, mapper, service.
// A new endpoint affects: handler, routes.
// A new external affects: externals generator, wire generator.
func (p *Planner) partialPlan(spec *ResolvedSpec, changes []SpecChange)
*GenerationPlan {
    affected := map[string]bool{}
    for _, change := range changes {
        for _, genID := range p.registry.AffectedBy(change) {
            affected[genID] = true
        }
    }
    return p.buildPlan(spec, affected, changes)
}

```

3.5 DAG Building and Topological Sort

This is where the dependency ordering is actually computed. The `GenerationPlan` holds a flat list of `Tasks`, each with a `DependsOn` field. The DAG builder constructs a directed acyclic graph from those edges and runs Kahn's algorithm to produce a topologically sorted list of tiers. Each tier is a group of tasks with no dependency on each other — they are safe to run in parallel.

Step 1 — Declare generator dependencies

Every generator declares which other generators it depends on. This is the source of truth for the DAG. It lives in a static dependency map inside the plan package — not inferred at runtime.

```

// internal/plan/dependencies.go

// generatorDeps maps generatorID → list of generatorIDs it depends on.
// A generator may only run after ALL its dependencies have completed.
var generatorDeps = map[string][]string{
    // tier 1 — no dependencies
    "sql.migration": {},
    "go.types":      {},
    "go.errors":     {},

    // tier 2 — depend on types and errors
    "go.model": {"go.types", "go.errors"},
    "go.dto":   {"go.types", "go.errors"},

    // tier 3 — depend on model and dto

```

```

    "go.repo":    {"go.model"},
    "go.mapper": {"go.model", "go.dto"},

    // tier 4 – depend on repo and mapper
    "go.service": {"go.repo", "go.mapper"},
    "go.auth":    {"go.repo", "go.mapper"},
    "go.external": {"go.errors"},
    "go.jobs":    {"go.repo", "go.service"},
    "go.tx":      {"go.repo"},

    // tier 5 – depend on service and dto
    "go.handler": {"go.service", "go.dto"},
    "go.routes":  {"go.handler", "go.auth"},

    // tier 6 – depend on everything
    "go.wire": {"go.service", "go.auth", "go.external", "go.jobs", "go.tx"},
    "go.mod":  {"go.wire"},
}

```

Step 2 — Build the adjacency list (DAG)

The DAG is an adjacency list built from the dependency map. Each node is a generator ID. Each directed edge $A \rightarrow B$ means "A must complete before B can start."

```

// internal/plan/dag.go

type DAG struct {
    nodes    map[string]struct{}
    edges    map[string][]string // node → list of nodes that depend on it
    inDegree map[string]int   // node → number of unmet dependencies
}

// buildDAG constructs the graph from the static dependency map,
// filtered to only the generators included in this generation plan.
func buildDAG(tasks []Task) (*DAG, error) {
    dag := &DAG{
        nodes:    map[string]struct{}{},
        edges:    map[string][]string{},
        inDegree: map[string]int{},
    }

    // register all nodes – only generators in this plan
    for _, t := range tasks {
        dag.nodes[t.GeneratorID] = struct{}{}
        if _, exists := dag.inDegree[t.GeneratorID]; !exists {
            dag.inDegree[t.GeneratorID] = 0
        }
    }

    // add edges from dependency declarations
    for _, t := range tasks {
        deps := generatorDeps[t.GeneratorID]
        for _, dep := range deps {
            // only add edge if the dependency is also in this plan
            // (partial plans skip generators not affected by the change)
            if _, inPlan := dag.nodes[dep]; !inPlan {
                continue
            }
        }
    }
}

```

```

        dag.edges[dep] = append(dag.edges[dep], t.GeneratorID)
        dag.inDegree[t.GeneratorID]++
    }
}

return dag, nil
}

```

Step 3 — Kahn's algorithm produces the tier list

Kahn's algorithm processes the DAG and produces an ordered list of tiers. It starts with all nodes that have zero in-degree (no unmet dependencies), processes them as a tier, then reduces the in-degree of their dependents. Any node whose in-degree reaches zero joins the next tier. If the algorithm completes with nodes remaining, there is a cycle — which means a bug in the dependency declarations.

```

// internal/plan/dag.go (continued)

// TopologicalTiers runs Kahn's algorithm on the DAG.
// Returns tiers in execution order — tier[0] runs first.
// Each tier is a []Task safe to run in parallel.
// Returns error if a cycle is detected (should never happen in practice).
func (d *DAG) TopologicalTiers(tasksByID map[string]Task) ([][]Task, error) {
    // queue starts with all nodes that have no dependencies
    queue := []string{}
    for node := range d.nodes {
        if d.inDegree[node] == 0 {
            queue = append(queue, node)
        }
    }
    sort.Strings(queue) // deterministic ordering within a tier

    tiers := [][]Task{}
    processed := 0

    for len(queue) > 0 {
        // current queue = one complete tier
        tier := []Task{}
        for _, nodeID := range queue {
            tier = append(tier, tasksByID[nodeID])
            processed++
        }
        tiers = append(tiers, tier)

        // compute next tier: reduce in-degree of all dependents
        nextQueue := []string{}
        for _, nodeID := range queue {
            for _, dependent := range d.edges[nodeID] {
                d.inDegree[dependent]--
                if d.inDegree[dependent] == 0 {
                    nextQueue = append(nextQueue, dependent)
                }
            }
        }
        sort.Strings(nextQueue)
        queue = nextQueue
    }
}

```

```

    // cycle detection: if processed < total nodes, there is a cycle
    if processed < len(d.nodes) {
        cycle := findCycle(d) // walk graph to identify the cycle
        return nil, fmt.Errorf("dependency cycle detected: %s", cycle)
    }

    return tiers, nil
}

```

Step 4 — Full buildPlan flow

Putting it all together — this is what the Planner calls to produce the final GenerationPlan from the list of affected generator IDs.

```

// internal/plan/planner.go (continued)

func (p *Planner) buildPlan(
    spec *ResolvedSpec,
    affected map[string]bool,
    changes []SpecChange,
) *GenerationPlan {

    // Step A: for each affected generator, build a Task per resource
    tasks := []Task{}
    tasksByID := map[string]Task{}

    for genID := range affected {
        gen := p.registry.Get(genID)
        if gen.IsPerResource() {
            // resource-scoped generators run once per affected resource
            for _, resource := range spec.Resources {
                if !resourceAffected(resource, changes) { continue }
                taskID := genID + ":" + resource.Name
                t := Task{
                    ID:          taskID,
                    GeneratorID: genID,
                    Context: GeneratorContext{
                        Spec:    spec,
                        Resource: &resource,
                        Changes: changes,
                    },
                }
                tasks = append(tasks, t)
                tasksByID[taskID] = t
            }
        } else {
            // cross-cutting generators run once (wire, mod, migration)
            t := Task{
                ID:          genID,
                GeneratorID: genID,
                Context: GeneratorContext{Spec: spec, Changes: changes},
            }
            tasks = append(tasks, t)
            tasksByID[genID] = t
        }
    }
}

```

```
// Step B: build DAG from task list
dag, err := buildDAG(tasks)
if err != nil { panic(err) } // cycle = programmer error, not user error

// Step C: topological sort → tiers
tiers, err := dag.TopologicalTiers(tasksByID)
if err != nil { panic(err) }

return &GenerationPlan{
    Tasks: tasks,
    Tiers: tiers,
    Reason: planReason(changes),
}
}
```

Walkthrough — adding a field to User

Concrete example: the developer adds a phone field to User. Here is exactly how the DAG builds and executes.

Step	What happens	Detail
1	Diff detects change	SpecChange{Kind: AddField, Target: "User.phone"} produced by diffSpecVsLock
2	Registry lookup	AffectedBy(AddField) returns [sql.migration, go.model, go.dto, go.repo, go.mapper, go.service, go.handler, go.wire, go.mod]
3	Task creation	Per-resource generators create task "go.model:User", "go.service:User" etc. Cross-cutting create "sql.migration", "go.wire"
4	DAG construction	Edges built from generatorDeps — only for generators in the affected set. go.model:User has in-degree 2 (depends on go.types, go.errors)
5	Kahn's algorithm	Tier 1: [sql.migration, go.types, go.errors] — all zero in-degree. Tier 2: [go.model:User, go.dto:User]. And so on.
6	Parallel execution	Orchestrator runs tier 1 in parallel (3 goroutines), waits, runs tier 2 in parallel (2 goroutines), and so on.
7	Files staged + flushed	All generated files collected in memory, written atomically to disk on Flush(). Lock file updated.

Dependency tiers — how parallel execution is determined

The planner groups tasks into tiers based on their DependsOn graph. All tasks in the same tier run in parallel. Tasks in later tiers run only after all prior tiers complete.

Tier	Generators	Why this tier
1	sql.migration, go.types, go.errors	No dependencies. Migration needs nothing. Types and errors are leaf nodes.
2	go.model, go.dto	Depends on types and errors from tier 1. Independent of each other.
3	go.repo, go.mapper	Repo depends on model. Mapper depends on model and dto.
4	go.service, go.auth, go.external, go.jobs, go.tx	All depend on repo and mapper. Independent of each other.
5	go.handler, go.routes	Depends on service and dto.
6	go.wire, go.mod	Depends on all services — must run last.

4. Generator System

4.1 Generator Interface

Every generator implements two methods. The interface is intentionally minimal — generators are pure functions from context to files.

```
// internal/generator/generator.go

type Generator interface {
    // ID returns the unique generator identifier used in task routing.
    // Convention: "{lang}.{concern}" e.g. "go.service", "sql.migration"
    ID() string

    // Generate receives a context containing the full resolved spec
    // (or just the relevant resource) and returns the files to write.
    // Generate must be pure — no file I/O, no global state, no network.
    Generate(ctx GeneratorContext) ([]File, error)
}

// GeneratorContext is everything a generator needs.
// Generators must not access anything outside this struct.
type GeneratorContext struct {
    Spec      *spec.ResolvedSpec
    Resource   *spec.ResolvedResource // nil for cross-cutting generators
    Changes    []diff.SpecChange       // empty on first run
    OutputDir  string
    Lang       spec.Lang
}
```

4.2 Registry

The registry holds all registered generators and knows which generators are affected by which change types. This is the routing table that the diff planner queries.

```
// internal/generator/registry.go

type Registry struct {
    generators map[string]Generator
    // affinity maps ChangeKind → list of generator IDs to re-run
    affinity   map[diff.ChangeKind][]string
}

func NewRegistry() *Registry {
    r := &Registry{
        generators: map[string]Generator{},
        affinity:   map[diff.ChangeKind][]string{},
    }
    return r
}

func (r *Registry) Register(g Generator, affects []diff.ChangeKind) {
```

```

    r.generators[g.ID()] = g
    for _, kind := range affects {
        r.affinity[kind] = append(r.affinity[kind], g.ID())
    }
}

func (r *Registry) AffectedBy(change diff.SpecChange) []string {
    return r.affinity[change.Kind]
}

// Registration happens in main — all generators registered at startup.
func RegisterAll(r *Registry, lang spec.Lang) {
    r.Register(&MigrationGenerator{}, []diff.ChangeKind{
        diff.AddField, diff.RemoveField, diff.AddResource,
        diff.AddRelation, diff.ChangeFieldType,
    })
    r.Register(&GoModelGenerator{}, []diff.ChangeKind{
        diff.AddField, diff.RemoveField, diff.AddResource, diff.ChangeFieldType,
    })
    r.Register(&GoDTOGenerator{}, []diff.ChangeKind{
        diff.AddField, diff.RemoveField, diff.AddDTO, diff.ChangeDTO,
    })
    // ... all other generators
}

```

4.3 Orchestrator

Runs the generation plan. Groups tasks into tiers by dependency order. Runs each tier in parallel using goroutines. Fails fast if any task errors — later tiers do not run.

```

// internal/generator/orchestrator.go

type Orchestrator struct {
    registry *Registry
    emitter  *emitter.Emitter
}

func (o *Orchestrator) Run(plan *plan.GenerationPlan) error {
    tiers := plan.TopologicalTiers()

    for tierIdx, tier := range tiers {
        var wg sync.WaitGroup
        errCh := make(chan error, len(tier))

        for _, task := range tier {
            wg.Add(1)
            go func(t plan.Task) {
                defer wg.Done()
                gen, ok := o.registry.Get(t.GeneratorID)
                if !ok {
                    errCh <- fmt.Errorf("unknown generator: %s", t.GeneratorID)
                    return
                }
                files, err := gen.Generate(t.Context)
                if err != nil {
                    errCh <- fmt.Errorf("generator %s: %w", t.GeneratorID, err)
                }
            }(task)
        }

        wg.Wait()
        if errCh != nil {
            return <errCh>
        }
    }
}

```

```
        return
    }
    for _, f := range files {
        if err := o.emitter.Stage(f); err != nil {
            errCh <- err
        }
    }
    }(task)
}
wg.Wait()
close(errCh)

// collect tier errors - do not proceed to next tier if any failed
var tierErrors []error
for err := range errCh {
    tierErrors = append(tierErrors, err)
}
if len(tierErrors) > 0 {
    return errors.Join(tierErrors...)
}

// all tiers succeeded - flush staged files to disk
return o.emitter.Flush()
}
```

Staged writes

The emitter stages files in memory during generation. Only after all tiers succeed does it write to disk in a single `Flush()` call. This ensures the generated/ directory is never left in a partial state if any generator fails.

5. Import Management

Import management is handled by the `ImportCollector` — one per generator, one per file type. The collector walks the `ResolvedResource` and builds the exact `ImportSet` needed. After template rendering, `goimports` runs as a safety net on Go files.

5.1 ImportSet

```
// internal/imports/set.go

type ImportSet struct {
    stdlib []string // "context", "fmt", "time"
    internal []string // "orders-service/generated/types"
    external []string // "github.com/google/uuid"
    seen map[string]bool
}

func (s *ImportSet) Add(pkg string) {
    if s.seen[pkg] { return }
    s.seen[pkg] = true
    switch {
    case isStdlib(pkg):
        s.stdlib = append(s.stdlib, pkg)
    case strings.HasPrefix(pkg, s.module+"/"):
        s.internal = append(s.internal, pkg)
    default:
        s.external = append(s.external, pkg)
    }
}

// Render produces the import block string for template insertion.
func (s *ImportSet) Render() string { /* grouped: stdlib / internal / external */ }
```

5.2 ImportCollector — per file type

Each generator has a dedicated collector function for each file it produces. The collector is a pure function — given a `ResolvedResource`, it deterministically returns the `ImportSet`.

```
// internal/imports/collectors.go

func ServiceImports(r spec.ResolvedResource, mod string) ImportSet {
    s := NewImportSet(mod)
    s.Add("context")
    s.Add("fmt")

    for _, field := range r.Fields {
        switch field.Type.Kind {
        case spec.TypeTimestamp: s.Add("time")
        case spec.TypeUUID:      s.Add("github.com/google/uuid")
        case spec.TypeJSON:      s.Add("encoding/json")
        case spec.TypeCustom:    s.Add(mod + "/generated/types")
        }
    }
}
```

```

    }
}
for _, rule := range r.Rules.AllRules() {
    switch rule.Type {
    case spec.RuleHash: s.Add("golang.org/x/crypto/bcrypt")
    }
}
if r.HasAfterRules() {
    s.Add(mod + "/generated/messaging")
}
if len(r.ExternalDeps) > 0 {
    s.Add(mod + "/generated/externals")
}
if r.HasSoftDelete {
    s.Add("database/sql")
}
return s
}

func DTOImports(dto spec.ResolvedDTO, mod string) ImportSet {
    s := NewImportSet(mod)
    for _, field := range dto.Fields {
        switch field.Type.Kind {
        case spec.TypeDecimal:    s.Add("math/big")
        case spec.TypeUUID:       s.Add("github.com/google/uuid")
        case spec.TypeTimestamp:  s.Add("time")
        case spec.TypeCustom:     s.Add(mod + "/generated/types")
        }
        if field.Required { s.Add("github.com/go-playground/validator/v10") }
    }
    return s
}

```

5.3 Post-render processing

After every Go template render, the output goes through two passes before becoming a File.

Pass	What it does
goimports	Adds any missing import, removes unused imports, groups in stdlib/internal/external order. If goimports fails the file has a syntax error — treated as a generator bug.
gofmt	Formats the file to canonical Go style. Indentation, spacing, brace placement. Template whitespace does not matter — gofmt fixes it.

```

// internal/template/postprocess.go

func PostProcessGo(filename string, src []byte) ([]byte, error) {
    // goimports = gofmt + import management
    out, err := imports.Process(filename, src, &imports.Options{
        Comments: true,
        TabIndent: true,
        TabWidth: 4,
    })
}

```

```
        if err != nil {
            // include original source in error for debugging
            return nil, fmt.Errorf("goimports failed on %s:\n%w\n---\n%s", filename,
err, src)
        }
        return out, nil
    }

    func PostProcessJava(filename string, src []byte) ([]byte, error) {
        // google-java-format via subprocess
        cmd := exec.Command("google-java-format", "-")
        cmd.Stdin = bytes.NewReader(src)
        out, err := cmd.Output()
        if err != nil { return src, nil } // java formatter is optional
        return out, nil
    }
```

6. Template Engine

6.1 Template loading

Templates are embedded in the binary using Go's embed package. They are loaded once at startup into a `template.Template` pool. Each generator retrieves its templates by name from this pool.

```
// internal/template/engine.go

//go:embed ../../templates
var templateFS embed.FS

type Engine struct {
    pool *template.Template
}

func NewEngine() (*Engine, error) {
    pool := template.New("").Funcs(buildFuncMap())
    // load all *.tmpl files recursively from embedded FS
    err := fs.WalkDir(templateFS, "templates", func(path string, d fs.DirEntry, err
error) error {
        if d.IsDir() || !strings.HasSuffix(path, ".tmpl") { return nil }
        data, _ := templateFS.ReadFile(path)
        template.Must(pool.New(path).Parse(string(data)))
        return nil
    })
    return &Engine{pool: pool}, err
}

func (e *Engine) Render(templateName string, data interface{}) ([]byte, error) {
    var buf bytes.Buffer
    if err := e.pool.ExecuteTemplate(&buf, templateName, data); err != nil {
        return nil, err
    }
    return buf.Bytes(), nil
}
```

6.2 FuncMap — template helper functions

The `FuncMap` provides all helper functions available inside templates. These handle naming conventions, type mapping, and language-specific formatting. All `FuncMap` functions are pure — no side effects.

Function	Purpose
<code>toSnakeCase</code>	"UserProfile" → "user_profile" — used for table and column names
<code>toCamelCase</code>	"user_id" → "userID" — Go field names follow Go conventions

toPascalCase	"user_profile" → "UserProfile" — type names, method names
toPlural	"User" → "users", "Address" → "addresses" — table names
typeMap	(dslType, lang) → language type string. "str", "go" → "string"
dbType	dslType → DB column type. "str" → "VARCHAR(255)"
zeroValue	lang type → zero value. "string" → `""`, "*User" → "nil"
isPointer	true if field is nullable — needs pointer type in Go
joinComma	[]string → comma-separated string for param lists
hasFeature	(resource, feature) → bool. Checks if resource uses bcrypt, uuid, etc.
renderImports	ImportSet → formatted import block string

6.3 Template data structs

Each generator builds a dedicated template data struct before calling Render. Templates never receive the ResolvedSpec directly — they receive a purpose-built struct with exactly the data they need. This keeps templates simple and keeps generator logic in Go code where it can be tested.

```
// internal/generators/go/service_generator.go

type ServiceTemplateData struct {
    PackageName    string
    Imports        imports.ImportSet
    ResourceName   string
    Methods        []ServiceMethod
    HasSoftDelete  bool
}

type ServiceMethod struct {
    Name           string           // "CreateUser"
    RequestDTO     string           // "CreateUserRequest"
    ResponseDTO    string           // "UserResponse"
    RepoMethod     string           // "CreateUser"
    GeneratedSteps []CodeSnippet    // pre-rendered code for generated logic
    BeforeHook     *HookCall
    AfterHook      *HookCall
    AfterActions   []CodeSnippet
    ReturnsSlice   bool
}

func (g *ServiceGenerator) buildData(ctx GeneratorContext) ServiceTemplateData {
    resource := ctx.Resource
    methods := []ServiceMethod{}
    for _, ep := range resource.Endpoints {
        methods = append(methods, g.buildMethod(ep, resource))
    }
}
```

```
    return ServiceTemplateData{
      PackageName: resource.PackageName,
      Imports:     imports.ServiceImports(*resource, ctx.Spec.Module),
      ResourceName: resource.Name,
      Methods:     methods,
    }
  }
```

7. File Emitter

The emitter is the only component that touches the filesystem. All generators produce File values in memory. The emitter collects them, then writes in a single atomic Flush. If Flush fails midway, it rolls back by restoring from backup.

```
// internal/emitter/emitter.go

type Emitter struct {
    outputDir string
    staged    []File
    mu        sync.Mutex    // protects staged - multiple goroutines call Stage()
}

// Stage accepts a rendered file. Thread-safe.
func (e *Emitter) Stage(f File) error {
    e.mu.Lock()
    defer e.mu.Unlock()
    e.staged = append(e.staged, f)
    return nil
}

// Flush writes all staged files. If any write fails, rolls back.
func (e *Emitter) Flush() error {
    // 1. unlock existing generated/ (chmod 755)
    e.unlockOutputDir()

    // 2. backup existing generated/ to generated/.backup
    if err := e.backup(); err != nil { return err }

    // 3. write all staged files
    for _, f := range e.staged {
        if err := e.writeFile(f); err != nil {
            e.rollback() // restore from backup
            return err
        }
    }

    // 4. lock generated/ (chmod 444 on all files)
    e.lockOutputDir()

    // 5. update lock file
    return e.writeLockFile()
}

func (e *Emitter) writeFile(f File) error {
    fullPath := filepath.Join(e.outputDir, f.Path)
    if err := os.MkdirAll(filepath.Dir(fullPath), 0755); err != nil {
        return err
    }
    mode := fs.FileMode(0444) // read-only by default
    if !f.ReadOnly { mode = 0644 }
    return os.WriteFile(fullPath, f.Content, mode)
}
```

8. Lock File

The lock file (generated/.goforge.lock) is a JSON snapshot of the ResolvedSpec at the time of last generation. It is the source of truth for the diff planner. It is always committed to version control alongside generated code.

```
// internal/diff/lock.go

type LockFile struct {
    Version      string           // GoForge tool version that generated this
    GeneratedAt  time.Time
    SpecHash     string           // SHA256 of goforge.yaml - fast change detection
    Spec         ResolvedSpec     // full snapshot for deep diff
}

func ReadLock(path string) (*LockFile, error) {
    data, err := os.ReadFile(path)
    if os.IsNotExist(err) { return nil, nil } // nil = first run
    if err != nil { return nil, err }
    var lock LockFile
    return &lock, json.Unmarshal(data, &lock)
}

// Fast path: if SpecHash matches, nothing changed - skip deep diff.
func (l *LockFile) QuickCheck(currentSpecHash string) bool {
    return l.SpecHash == currentSpecHash
}
```

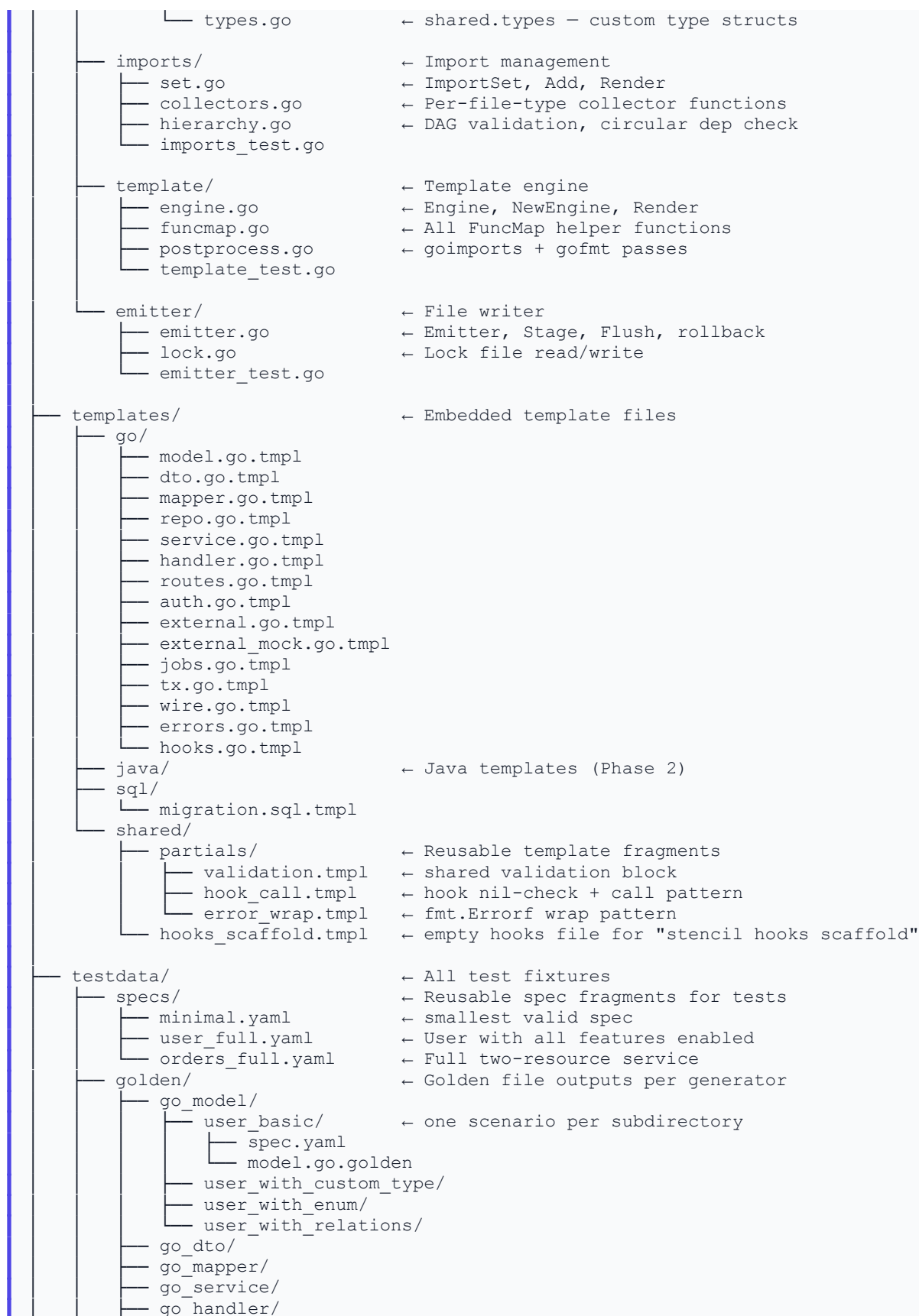
Lock file is generated code

The lock file lives inside generated/ and is written by the emitter on every successful run. It is read-only like all other generated files. Developers never edit it manually. If it gets corrupted, delete it and run goforge generate to recreate everything from scratch.

9. Folder Structure

The complete repository layout. Every directory has a single clear purpose. No file lives in an ambiguous location.







10. Testing Strategy

Testing a code generator is not like testing application logic. You are not testing whether a function returns the right number. You are testing that given a spec, the right source code comes out — and that source code actually works. This requires a layered strategy.

The core question for each test layer

Layer 1 (unit): does the pipeline produce correct internal data structures? Layer 2 (golden): does the generator produce the expected source text? Layer 3 (compile): does the generated source code actually build? Layer 4 (runtime): does the generated service actually behave correctly at HTTP level? Layer 5 (regression): does a spec change produce exactly the expected diff and nothing else?

10.1 Layer 1 — Unit tests (pipeline internals)

Tests for the Parser, Validator, and Resolver. These are fast table-driven tests that verify internal data structure transformations. They do not touch the filesystem or generate any code.

What to test

Component	Test cases
Parser	Valid YAML parses correctly. Malformed YAML returns ParseError with line number. Unknown top-level key produces warning not error. Every DSL section round-trips through parse.
Validator	Each of the 14 validation rules fires on a spec that violates it. Valid spec returns zero errors. All errors collected in single pass — no early return. Error codes are stable strings.
Resolver	table name inferred from resource name. Index derived from find_by. FK column added to correct resource. GoType/JavaType/DBType all resolved correctly per field type.
DAG	Correct tier grouping for full plan. Partial plan skips unaffected generators correctly. Cycle detection returns error with cycle path. Tier ordering is deterministic.
ImportSet	Deduplication works. Stdlib/internal/external grouping is correct. Render() produces correctly formatted import block.

10.2 Layer 2 — Golden file tests (generator output)

The primary test mechanism for generators. Each test feeds a known spec into a single generator and compares the output byte-for-byte against a committed golden file. Golden files are the source of truth for what the generator is supposed to produce.

How golden tests work

```
// Convention: testdata/golden/{generator}/{scenario}/spec.yaml → expected output files

func TestGoServiceGenerator(t *testing.T) {
    cases := []string{
        "user_basic",           // minimal User resource
        "user_with_bcrypt",     // has hash: password rule
        "user_with_events",     // has after: emit rules
        "user_with_states",     // has states block
        "user_with externals",  // depends on external client
        "user_with_tx",         // has transaction block
        "order_full",           // full Order resource
    }

    for _, tc := range cases {
        t.Run(tc, func(t *testing.T) {
            spec := loadSpec(t, "golden/go_service/"+tc+"/spec.yaml")
            gen := &GoServiceGenerator{engine: testEngine(t)}

            files, err := gen.Generate(GeneratorContext{
                Spec:      spec,
                Resource: &spec.Resources[0],
            })
            require.NoError(t, err)

            for _, f := range files {
                golden := loadGolden(t,
                    "golden/go_service/"+tc+"/"+filepath.Base(f.Path))
                if *update {
                    writeGolden(t, golden, f.Content) // -update flag
                } else {
                    assert.Equal(t, string(golden), string(f.Content),
                        "golden mismatch for %s in scenario %s", f.Path, tc)
                }
            }
        })
    }
}

var update = flag.Bool("update", false, "update golden files")
```

Golden file workflow — the dev loop

Step	Action	Detail
1	Write new scenario	Create testdata/golden/go_service/user_with_bcrypt/spec.yaml with the spec you want to test
2	Generate golden	go test ./... -run TestGoServiceGenerator -update — generates golden files from current output

3	Review in git	git diff shows exact generated code. This is your code review step — read every line of the generated output
4	Commit	Commit spec.yaml + golden files together. Golden files are source of truth from this point
5	Future changes	If template changes cause golden mismatch, test fails. Review diff, decide if change is correct, re-run -update if yes

Golden files are the code review

When a template change causes golden files to change, that diff is the entire review surface. You can see exactly what changed in the generated output across every scenario. This is more valuable than reading the template change itself — you see the effect, not the cause.

10.3 Layer 3 — Compile test

Takes a complete spec, runs the full pipeline end to end, then runs the language compiler on the output. This catches cases where individual generators produce valid-looking code that does not compile when assembled together — missing imports, type mismatches between generated files, incorrect function signatures.

```
// e2e/compile_test.go

func TestGeneratedGoCodeCompiles(t *testing.T) {
    if testing.Short() { t.Skip("compile test skipped in short mode") }

    cases := []struct {
        name string
        spec string
    }{
        {"minimal service", "testdata/integration/minimal_service/stencil.yaml"},
        {"orders service", "testdata/integration/orders_service/stencil.yaml"},
    }

    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel()
            outDir := t.TempDir()

            err := stencil.Generate(tc.spec, outDir)
            require.NoError(t, err, "generation failed")

            // go build ./... on the generated output
            cmd := exec.Command("go", "build", "./...")
            cmd.Dir = outDir
            out, err := cmd.CombinedOutput()
            require.NoError(t, err,
                "generated code failed to compile:\n%s", string(out))

            // go vet on generated output
```

```

        cmd = exec.Command("go", "vet", "./...")
        cmd.Dir = outDir
        out, err = cmd.CombinedOutput()
        require.NoError(t, err,
            "go vet failed on generated code:\n%s", string(out))
    })
}
}

```

10.4 Layer 4 — Runtime test (the real integration test)

The most valuable test and the hardest to set up. Takes the full `orders_service` spec, generates the complete service, starts it against a real Postgres database, then hits the HTTP endpoints and asserts correct behaviour. This is the only layer that verifies the generated service actually works at runtime.

Setup

```

# e2e/runtime_test.go uses docker-compose to spin up postgres
# Runs as a separate make target: make test-runtime
# Not run in CI on every commit — runs nightly and on release branches

func TestRuntimeBehaviour(t *testing.T) {
    if os.Getenv("STENCIL_RUNTIME_TESTS") == "" {
        t.Skip("set STENCIL_RUNTIME_TESTS=1 to run")
    }

    // Step 1: generate the service
    outDir := t.TempDir()
    err := stencil.Generate("testdata/integration/orders_service/stencil.yaml",
outDir)
    require.NoError(t, err)

    // Step 2: run migrations against test postgres
    runMigrations(t, outDir, testDSN)

    // Step 3: start the generated service
    svc := startService(t, outDir, testDSN)
    defer svc.Stop()

    // Step 4: hit the HTTP endpoints
    client := &http.Client{}
    baseURL := svc.URL()

    // --- Create user ---
    resp := post(t, client, baseURL+"/users", map[string]any{
        "first_name": "Harshit",
        "last_name":  "Patel",
        "email":      "harshit@example.com",
        "password":   "securepassword123",
    })
    assert.Equal(t, 201, resp.StatusCode)
    var user map[string]any
    decodeJSON(t, resp, &user)
    assert.Equal(t, "Harshit", user["first_name"])
}

```

```

    assert.NotContains(t, user, "password_hash", "private field leaked")

    // --- Login ---
    resp = post(t, client, baseURL+"/auth/login", map[string]any{
        "email":    "harshit@example.com",
        "password": "securepassword123",
    })
    assert.Equal(t, 200, resp.StatusCode)
    var auth map[string]any
    decodeJSON(t, resp, &auth)
    token := auth["token"].(string)
    assert.NotEmpty(t, token)

    // --- Create order (authenticated) ---
    resp = postAuthed(t, client, baseURL+"/orders", token, map[string]any{
        "items": []map[string]any{{"product_id": 1, "quantity": 2}},
    })
    assert.Equal(t, 201, resp.StatusCode)

    // --- Verify soft delete ---
    userID := user["id"].(string)
    resp = deleteAuthed(t, client, baseURL+"/users/"+userID, adminToken)
    assert.Equal(t, 204, resp.StatusCode)

    // user should 404 now
    resp = getAuthed(t, client, baseURL+"/users/"+userID, token)
    assert.Equal(t, 404, resp.StatusCode)

    // but should still exist in DB (soft deleted, not hard deleted)
    assertRowExists(t, testDSN, "SELECT 1 FROM users WHERE id=$1 AND deleted_at IS
NOT NULL", userID)
}

```

What the runtime test verifies that no other layer can

Behaviour	Why only runtime test catches it
Private fields never leak	Compile test does not send HTTP requests. Only runtime test inspects actual JSON response bodies.
Auth middleware fires correctly	Compile test does not start the server. Only runtime test verifies 401 on missing token, 403 on wrong role.
Soft delete filters work	Only runtime test queries through the HTTP layer after a delete and verifies 404, then checks DB directly for the row.
Password is hashed in DB	Only runtime test can login with plain text password and assert the DB contains a hash, not plain text.
State machine blocks invalid tx	Only runtime test attempts an invalid state transition and verifies 422 with the right error code.
Pagination works end to end	Only runtime test creates N records then verifies cursor pagination returns correct pages in correct order.
Migrations ran correctly	Only runtime test verifies DB schema matches what the generated service expects — column types, indexes, FK constraints.

10.5 Layer 5 — Update regression test

Verifies that running stencil update after a spec change produces exactly the expected diff — the right files changed, and no files that should not have changed were touched. This is the test that protects the update contract.

```
// e2e/update_test.go

func TestUpdateRegression(t *testing.T) {
    cases := []struct {
        name          string
        baseSpec       string // spec before the change
        updatedSpec    string // spec after the change
        expectChanged []string // files that MUST differ
        expectUnchanged []string // files that MUST NOT change
    }{
        {
            name:          "add field to User",
            baseSpec:       "testdata/update/add_field/before.yaml",
            updatedSpec:    "testdata/update/add_field/after.yaml",
            expectChanged: []string{
                "generated/user/model.go",
                "generated/user/dto.go",
                "generated/user/mapper.go",
                "generated/user/repository.go",
                "generated/migration/002_add_phone_to_users.sql",
            },
            expectUnchanged: []string{
                "generated/user/service.go", // field add does not change service
                "generated/order/model.go", // unrelated resource untouched
                "generated/wire.go",           // DI unchanged
                "hooks/user/user.hooks.go",    // developer file NEVER touched
            },
        },
        {
            name:          "add endpoint to Order",
            baseSpec:       "testdata/update/add_endpoint/before.yaml",
            updatedSpec:    "testdata/update/add_endpoint/after.yaml",
            expectChanged: []string{
                "generated/order/handler.go",
                "generated/order/routes.go",
                "generated/order/dto.go",
            },
            expectUnchanged: []string{
                "generated/order/model.go",
                "generated/order/repository.go",
                "generated/user/model.go",
            },
        },
    },

    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            outDir := t.TempDir()

            // generate from base spec
```

```

    err := stencil.Generate(tc.baseSpec, outDir)
    require.NoError(t, err)

    // snapshot file hashes before update
    before := hashDir(t, outDir)

    // run update with changed spec
    err = stencil.Update(tc.updatedSpec, outDir)
    require.NoError(t, err)

    // snapshot after
    after := hashDir(t, outDir)

    // assert expected files changed
    for _, path := range tc.expectChanged {
        assert.NotEqual(t, before[path], after[path],
            "expected %s to change but it did not", path)
    }

    // assert untouched files are identical
    for _, path := range tc.expectUnchanged {
        assert.Equal(t, before[path], after[path],
            "expected %s to be unchanged but it was modified", path)
    }
})
}

```

10.6 Layer 6 — Idempotency test

Running generate twice on an unchanged spec must produce byte-identical output. This is a hard requirement — if it fails, the diff planner has a bug or a template has non-deterministic output (e.g. map iteration order).

```

func TestIdempotency(t *testing.T) {
    specs := []string{
        "testdata/integration/minimal_service/stencil.yaml",
        "testdata/integration/orders_service/stencil.yaml",
    }
    for _, spec := range specs {
        t.Run(spec, func(t *testing.T) {
            dir1, dir2 := t.TempDir(), t.TempDir()
            require.NoError(t, stencil.Generate(spec, dir1))
            require.NoError(t, stencil.Generate(spec, dir2))

            diff := diffDirs(dir1, dir2)
            assert.Empty(t, diff,
                "generate is not idempotent - diff:\n%s", diff)
        })
    }
}

```

10.7 Manual testing workflow — the dev loop

This is how you test during active development on a generator or template. No CI, no test commands — just running the tool against a real spec and looking at the output.

The loop

Step	Command	What you do
1	stencil validate stencil.yaml	Fix any spec errors first. Validate catches all errors in one pass — read the full list before changing anything.
2	stencil diff stencil.yaml	See what will be generated or changed without writing anything. Understand the plan before running it.
3	stencil generate stencil.yaml	Generate into the target directory. Then open generated/ and read the files. Read them like a code review.
4	cd output && go build ./...	Does it compile? If not, the error points to exactly which generated file and line has the problem.
5	go vet ./...	Catch semantic issues the compiler allows but are likely bugs — unreachable code, printf format mismatches.
6	golangci-lint run	Full lint pass. Generated code must pass with zero warnings — same bar as hand-written code.
7	Read the hooks scaffold	Run stencil hooks scaffold User. Open hooks/user/user.hooks.go. Is the interface obvious? Is it easy to implement?
8	Implement a hook, run the service	Implement one hook (e.g. BeforeCreate). Start the service with go run ./cmd/server. Hit the endpoint with curl. Does it behave?

What to look for when reading generated files

- model.go — are all fields present? correct Go types? correct json tags? private fields absent from json?
- dto.go — does CreateRequest have the right fields? is password in plain text not hash? does UpdateRequest have optional fields?
- mapper.go — does FromCreateRequest exclude password? does ToResponse exclude private fields? are computed fields commented?
- service.go — are hook call sites in the right order? do generated steps (hash, unique check) appear before the DB call? do after actions appear after commit?
- handler.go — does each endpoint use the right DTO? does it return the right status code (201 for create, 204 for delete)?
- hooks.go — is every hook point present? are signatures typed correctly? would a developer know what to implement from reading this file alone?

- migration.sql — are all columns present? correct types? indexes generated? FK constraints correct?

Makefile targets

```
# Fast — runs every commit
make test          # unit tests + golden file tests (no filesystem)
make lint          # golangci-lint on GoForge tool itself

# Medium — runs on PR merge
make test-compile  # generate + go build + go vet on integration specs
make test-update   # update regression tests
make test-idempotency

# Slow — runs nightly
make test-runtime  # full HTTP-level runtime tests against real Postgres

# Development
make golden-update # regenerate all golden files (review diff before committing)
make dev-generate  # generate from ./stencil.yaml into ./dev-output/
make dev-compile   # go build ./... on ./dev-output/

# Release
make test-all      # all layers in sequence
make build          # cross-compile binaries for all platforms
```

11. Adding a New Generator

This section is a step-by-step guide for contributors adding a new generator to GoForge. Follow all steps in order.

Step 1 — Define the generator struct

```
// internal/generators/go/my_generator.go

type MyGenerator struct {
    engine *template.Engine
}

func (g *MyGenerator) ID() string { return "go.my" }

func (g *MyGenerator) Generate(ctx generator.GeneratorContext) ([]generator.File,
error) {
    // build template data
    // render template
    // post-process (goimports)
    // return []File
}
```

Step 2 — Write the template

```
// templates/go/my.go.tmpl

package {{ .PackageName }}

import (
    {{ renderImports .Imports }}
)

// ... template body
```

Step 3 — Write the ImportCollector function

```
// internal/imports/collectors.go - add new function

func MyImports(r spec.ResolvedResource, mod string) ImportSet {
    s := NewImportSet(mod)
    // add imports based on what the generator uses
    return s
}
```

Step 4 — Register with affected change kinds

```
// cmd/goforge/main.go - inside RegisterAll()

r.Register(&MyGenerator{engine: engine}, []diff.ChangeKind{
    diff.AddField,
    diff.AddResource,
    // add all change kinds that should trigger this generator
})
```

Step 5 — Declare dependency tier

```
// internal/plan/tiers.go - add to the correct tier

var generatorTiers = [][]string{
    {"sql.migration", "go.types", "go.errors"}, // tier 1
    {"go.model", "go.dto"}, // tier 2
    {"go.repo", "go.mapper"}, // tier 3
    {"go.service", "go.auth", "go.my"}, // tier 4 ← add here
    {"go.handler", "go.routes"}, // tier 5
    {"go.wire", "go.mod"}, // tier 6
}
```

Step 6 — Write golden file tests

```
// testdata/generators/go_my/basic/spec.yaml ← create test spec
// testdata/generators/go_my/basic/my.go.golden ← create expected output

// Run: go test ./... -update to generate golden files on first run.
// Review the generated golden file in git before committing.
```

Step 7 — Run the full test suite

```
make test          # unit + golden file tests
make test-slow     # includes compile test + idempotency test
make lint          # golangci-lint on GoForge itself
```